

VAANI AI — INTELLIGENT VOICE-FIRST ENGLISH COMMUNICATION & SPEECH COACHING PLATFORM

A project report submitted in fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

- STUDENT ENGINEER I (Reg No: 319126510001)
- STUDENT ENGINEER II (Reg No: 319126510002)

Under the esteemed guidance of

FACULTY ADVISOR NAME

(Assistant Professor)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES (A)

(UGC AUTONOMOUS)

(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA & NAAC with 'A' Grade)

SANGIVALASA, VISAKHAPATNAM-531162

2025–2026

**ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND
SCIENCES (A)**

(UGC AUTONOMOUS)

(Affiliated to AU, Approved by AICTE and Accredited by NBA & NAAC with 'A' Grade)

Sangivalasa, Bheemili Mandal, Visakhapatnam dist. (A.P)

CERTIFICATE

This is to certify that the project report entitled "**VAANI AI — INTELLIGENT VOICE-FIRST ENGLISH COMMUNICATION & SPEECH COACHING PLATFORM**" submitted by Student Engineer I (319126510001) and Student Engineer II (319126510002) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science**

Engineering of Anil Neerukonda Institute of Technology and Sciences (A), Visakhapatnam is a record of bonafide work carried out under my guidance and supervision.

Project Guide

Assistant Professor

Department of CSE, ANITS

Head of the Department

Professor

Department of CSE, ANITS

DECLARATION

We, Student Engineer I (319126510001) and Student Engineer II (319126510002), final semester B.Tech students in the Department of Computer Science and Engineering from ANITS, Visakhapatnam, hereby declare that the project work entitled **"VAANI AI — INTELLIGENT VOICE-FIRST ENGLISH COMMUNICATION & SPEECH COACHING PLATFORM"** carried out by us and submitted in partial fulfillment of the requirements for the award of **Bachelor of Technology in Computer Science Engineering**, under Anil Neerukonda Institute of Technology & Sciences(A) during the academic year 2025–2026, has not been submitted to any other university for the award of any kind of degree.

STUDENT ENGINEER I (319126510001)

STUDENT ENGINEER II (319126510002)

ACKNOWLEDGEMENT

We would like to express our deep gratitude to our project guide for his/her guidance with unsurpassed knowledge and immense encouragement. We are grateful to the Head of the Department, Computer Science and Engineering, for providing us with the required facilities for the completion of the project work.

We are very much thankful to the Principal and Management, ANITS, Sangivalasa, for their encouragement and cooperation to carry out this work.

We express our thanks to the Project Coordinator for his/her continuous support and encouragement. We thank all the teaching faculty of the Department of CSE, whose suggestions during reviews helped us in accomplishment of our project.

We would like to thank our parents, friends, and classmates for their encouragement throughout our project period. At last but not the least, we thank everyone for supporting us directly or indirectly in completing this project successfully.

PROJECT STUDENTS:

- STUDENT ENGINEER I (319126510001)
- STUDENT ENGINEER II (319126510002)

ABSTRACT

Traditional methods of diagnosing and evaluating spoken language proficiency rely mainly on human expert tutors, which easily causes financial barriers, scheduling delays, and subjective assessment. Due to the requirement for real-time speech interaction, low-latency audio transcription, and objective acoustic evaluation, a voice-first intelligent coaching system is required.

We propose **Vaani AI (ConviAI)**, a deep learning-based intelligent voice coaching platform engineered to evaluate spoken English fluency, pronunciation accuracy, grammatical correctness, and conversational performance. Vaani AI combines low-latency browser speech APIs (`webkitSpeechRecognition` STT and `window.speechSynthesis` TTS) with backend deep learning models. The backend architecture features **Wav2Vec2-base-960h** (a PyTorch speech transformer) for Connectionist Temporal Classification (CTC) phoneme alignment and Levenshtein pronunciation scoring, **Faster-Whisper-tiny.en** (an INT8 CTranslate2 quantized seq2seq model) for offline speech-to-text transcription, and a multi-tier LLM fallback gateway (**Gemini 3.5 Flash**, **Groq Llama-3 8B**, **Local Mistral 7B**, and heuristic regex processors) for dynamic grammar error correction (GEC) and adaptive conversational dialogue.

Empirical evaluation across a 36-group automated REST API test suite (`smoke_test.py`) demonstrates 100% endpoint reliability, sub-1.5-second boot times, and real-time audio analysis capability.

Keywords: Deep Learning, Speech Recognition, Wav2Vec2 CTC, Faster-Whisper, Large Language Models (LLM), Grammar Error Correction (GEC), FastAPI, Next.js, Data Flow Diagrams (DFD), Chen ER Model, UML Class Diagrams.

CONTENTS

- **Abstract** v
- **List of Figures** ix
- **List of Tables** x
- **List of Abbreviations** xi
- **Chapter 1: Introduction**
 - 1.1 Introduction 1
 - 1.2 Motivation of work 2
 - 1.3 Problem Statement 2
- **Chapter 2: Literature Survey**
 - 2.1 Introduction 3
 - 2.2 Classification and Speech Processing in CALL Systems 3
 - 2.3 Deep Learning for Voice-Based Language Learning 3
 - 2.4 Comparative analysis of speech and NLP algorithms 4

- **Chapter 3: Proposed Methodology**
- 3.1 Introduction 5
- 3.2 Objectives 5
- 3.3 System Architecture 5
- 3.3.1 Speech Processing Pipeline 6
- 3.3.2 Phoneme Alignment & CTC Logits 6
- 3.3.3 Parameters & Speech Metrics Setting 7
- 3.3.4 Multi-Tier LLM Gateway Initialisation 8
- 3.3.5 Dynamic Progressive Difficulty & Prompting 9
- 3.3.6 Language Immersion & Non-English Detection 9
- 3.3.7 Spoken Communication & Grammar Feedback 12
- **Chapter 4: Requirement Analysis**
- 4.1 Introduction 14
- 4.2 Functional Requirements 14
- 4.3 Non-functional requirements 14
- 4.4 Technical Requirements 15
- 4.4.1 Hardware Requirements 15
- 4.4.2 Software Requirements 15
- 4.5 Student / Developer requirements 16
- **Chapter 5: Modules division**
- 5.1 Authentication Module 17
- 5.2 Voice Practice & Speech Scoring Module 20
- 5.3 AI Speaking Partner Module 21
- 5.4 Daily Challenges & Mock Interview Module 22
- 5.5 Analytics & Progress Tracking Module 23
- 5.6 Detailed File-by-File Codebase Inventory & Analysis 24
- **Chapter 6: Model**
- 6.1 Wav2Vec2 Speech Transformer Model 25
- 6.1.1 Convolution Layers & Feature Extractor 25
- 6.1.2 Transformer Encoder & Attention Layers 26
- 6.1.3 CTC Classification Layer 28
- 6.2 Faster-Whisper Model (CTranslate2 INT8) 29
- 6.3 Multi-Tier LLM Gateway Model 33
- **Chapter 7: UML Diagrams**
- 7.1 Use Case Diagrams 36
- 7.1.1 Purpose of Use Case Diagrams 36
- 7.1.2 Use Case Diagram 37
- 7.2 DFD Diagrams 38
- 7.2.1 Symbols of DFDs 38
- 7.2.2 Levels in Data Flow Diagrams (DFD) 39
- 7.2.2.1 Level 0 DFD (Context Diagram) 39
- 7.2.2.2 Level 1 DFD 40
- 7.2.2.3 Level 2 DFD 41
- 7.3 Class Diagram 42

- 7.3.1 Purpose of Class Diagrams 42
- 7.3.2 Class Diagram 42
- 7.4 Entity-Relationship (ER) Diagram 43
- 7.4.1 Purpose of ER Diagrams 43
- 7.4.2 Chen Model ER Diagram 43
- **Chapter 8: Dataset**
- 8.1 Introduction 44
- 8.2 Database Architecture & Relational Tables 44
- 8.3 Pre-Populated GRE Vocabulary & Challenge Data 45
- **Chapter 9: Implementation and Code**
- 9.1 Environment & Dependency Installation 46
- 9.2 Backend FastAPI Implementation & Routing 47
- 9.3 Frontend Next.js Implementation & Web Speech 50
- 9.4 Containerization & Multi-stage Docker 51
- **Chapter 10: Performance Measures**
- 10.1 Precision & Levenshtein Alignment 53
- 10.2 Recall & Speech Fluency Metrics 54
- 10.3 Mean Average Accuracy & Confidence Score 56
- 10.4 F1 Score 57
- 10.5 Evaluation Summary 58
- **Chapter 11: Experimental Analysis and Results**
- 11.1 Results of Automated REST API Test Suite (`smoke_test.py`) 60
- 11.2 Results of Real-Time WebSocket Streaming (`ws_test.py`) 62
- 11.3 Performance Benchmarks & Latency Results 64
- **Chapter 12: Limitations**
- 12.1 Limitations 66
- **Chapter 13: Conclusion and Future scope**
- 13.1 Conclusion 67
- 13.2 Future scope 67
- **References** 68

LIST OF FIGURES

- **Fig 3.1:** Architecture of proposed system 5
- **Fig 3.2:** Flowchart of speech analysis algorithm 6
- **Fig 5.1:** Directory structure of Vaani AI workspace 17
- **Fig 6.1:** Internal architecture of Wav2Vec2 CTC Transformer 25
- **Fig 6.2:** Multi-Tier LLM Gateway Execution Architecture 30
- **Fig 7.1:** Use-Case Diagram 37
- **Fig 7.2.1:** Symbols of DFDs 38
- **Fig 7.2.2:** Data Flow Diagram (level 0) 39
- **Fig 7.2.3:** Data Flow Diagram (level 1) 40
- **Fig 7.2.4:** Data Flow Diagram (level 2) 41

- **Fig 7.3:** Class Diagram 42
- **Fig 7.4:** Entity-Relationship Diagram (Chen Model) 43
- **Fig 11.1:** Test Execution Results of Automated Smoke Test Suite 60

LIST OF TABLES

- **Table 2.1:** Comparison of different speech recognition and NLP models 4
- **Table 4.1:** Hardware Requirements 15
- **Table 4.2:** Software Requirements 15
- **Table 5.1:** Exhaustive File Inventory & Module Mapping 24
- **Table 8.1:** Relational Database Tables Specification 44
- **Table 9.1:** Complete REST API Endpoints Specification 48
- **Table 10.1:** System Latency & Resource Benchmarks 58
- **Table 11.1:** Summary Results of Automated Smoke Test Suite 61

LIST OF ABBREVIATIONS

- **AI:** Artificial Intelligence
- **API:** Application Programming Interface
- **ASR:** Automatic Speech Recognition
- **CALL:** Computer-Assisted Language Learning
- **CEFR:** Common European Framework of Reference for Languages
- **CORS:** Cross-Origin Resource Sharing
- **CTC:** Connectionist Temporal Classification
- **DFD:** Data Flow Diagram
- **DL:** Deep Learning
- **ER:** Entity-Relationship
- **GEC:** Grammar Error Correction
- **GD:** Group Discussion
- **JWT:** JSON Web Token
- **LLM:** Large Language Model
- **ML:** Machine Learning
- **NLP:** Natural Language Processing
- **REST:** Representational State Transfer
- **STT:** Speech-to-Text
- **TTS:** Text-to-Speech
- **UML:** Unified Modelling Language
- **WER:** Word Error Rate
- **WPM:** Words Per Minute

Chapter 1: Introduction

1.1. Introduction

Good communication is considered an essential part of professional and personal life. In order to excel in modern workplaces and academic environments, early mastery of spoken English fluency and pronunciation is crucial. Annual educational and professional losses due to communication barriers remain high across non-native speaking regions. Therefore, early automated evaluation and practice of spoken English is important.

One of the most effective solutions is to predict, evaluate, and classify speech performance by collecting audio growth inputs and building a voice-first intelligent coaching model, which can provide fast and effective alerts and feedback, improve speaking practice efficiency, and reduce speech errors. The traditional method of diagnosing speaking errors is mainly based on human expert tutors evaluating audio recordings manually. It directs language learning by manually observing pronunciation, grammar, vocabulary, and pace, leading to problems such as high costs, long diagnosis delays, and low tutor availability.

Building on this, researchers are working on evaluating spoken language using deep learning network models to achieve rapid transcription and classification of speech errors. Object detection and speech recognition technologies have become active research fields. In speech processing, models like Wav2Vec2 and Whisper allow real-time acoustic phoneme alignment and transcription. Ultralytics and Meta developed deep learning models which flexibly control model inference speed, memory footprint, and classification accuracy. Based on comparative performance analysis of different speech algorithms, this study proposes a voice-first coaching architecture suited for spoken English evaluation by combining client-side Web Speech APIs with PyTorch deep learning models and multi-tier LLM gateways.

1.2. Motivation of work

Speech errors cause reductions in speaking confidence, leading to professional limitations for students and job seekers. Moreover, relying solely on human tutors can introduce high recurring costs. Using deep learning techniques to evaluate speech can help in early detection and diagnosis of pronunciation and grammar errors, allowing for timely feedback and practice measures. Deep learning models can analyze audio waveforms, transcribing speech accurately and scoring pronunciation based on acoustic alignment. This aids precision language coaching, where learners receive targeted corrections only on affected words, improving learning efficiency and minimizing speech anxiety.

1.3. Problem Statement

Due to high latency and computational overhead in traditional cloud-based voice engines, real-time speech coaching accuracy and speed are required to be high. Therefore, it is necessary to optimize and integrate existing deep learning models (Wav2Vec2, Faster-Whisper, LLMs, and LanguageTool) to meet detection needs. So, we came up with a deep learning-based approach to identify speech errors and classify pronunciation and grammar quality using PyTorch acoustic models and FastAPI server architecture.

Chapter 2: Literature Survey

2.1. Introduction

Spoken language assessment has been a major concern for education, leading to professional barriers. Accurate and timely detection of pronunciation and grammatical mistakes is crucial to improve fluency. In recent years, there has been growing interest in using deep learning approaches to classify speech errors.

2.2. Classification and Speech Processing in CALL Systems

S. Arivazhagan et al. proposed methods for automated feature extraction from acoustic signals. Researchers developed processing schemes consisting of color/acoustic transformation, threshold masking, and feature extraction for classifiers. While achieving good baseline accuracy, traditional feature extraction relies on small sample sizes and lacks contextual deep learning representations.

2.3. Deep Learning for Voice-Based Language Learning

Sharada P. Mohanty et al. demonstrated deep learning for automated image and signal classification using convolutional networks across 14 species and 26 classes, reaching over 99% accuracy on held-out test sets. However, when tested on unseen conditions, model accuracy decreased substantially. Applying transformer-based self-supervised models like Wav2Vec2 and Whisper resolves generalization limits by training on thousands of hours of diverse speech audio.

2.4. Comparative analysis of deep learning speech and NLP algorithms

Shrey Srivastava et al. performed comparative analyses of deep learning algorithms (SSD, Faster R-CNN, YOLO, Transformers). In identical testing environments, Transformer-based acoustic and language models outperform traditional HMM and CNN models, making them the best choice for speech coaching.

Table 2.1: Comparison of different speech recognition and NLP models

Model	Generation Time	Network-Level	Structural Features	Advantages and Limitations
GMM-HMM	1998	5	Gaussian acoustic state modeling	Poor processing power for continuous non-native speech
CNN-LSTM	2014	19	Recurrent frame memory layers	High memory usage, latency in long audio
Wav2Vec2	2020	12 (Transformer)	CTC self-supervised frame alignment	High precision phoneme scoring, requires

				explicit transcript
Faster-Whisper	2023	Encoder-Decoder	INT8 CTranslate2 quantization	Ultra-fast STT execution, low memory footprint
Gemini 3.5 Flash	2024	Decoder LLM	Multimodal cloud LLM gateway	High reasoning speed, requires internet connectivity

Chapter 3: Proposed Methodology

3.1. Introduction

Spoken language errors are a threat to career growth. With advancements in computer vision and speech deep learning, it has become possible to identify pronunciation and grammar errors at an early stage. In this proposed methodology, we aim to evaluate spoken speech using Wav2Vec2 CTC acoustic models and multi-tier LLM gateways.

3.2. Objectives

The objectives of this proposed system are as follows:

- Detect pronunciation and grammatical speech errors in real time.
- Identify symptoms of non-native language interference (e.g., Hindi transliteration) at an early stage.
- Accurately score pronunciation, pace (WPM), confidence, and grammar rewrites.

3.3. System Architecture

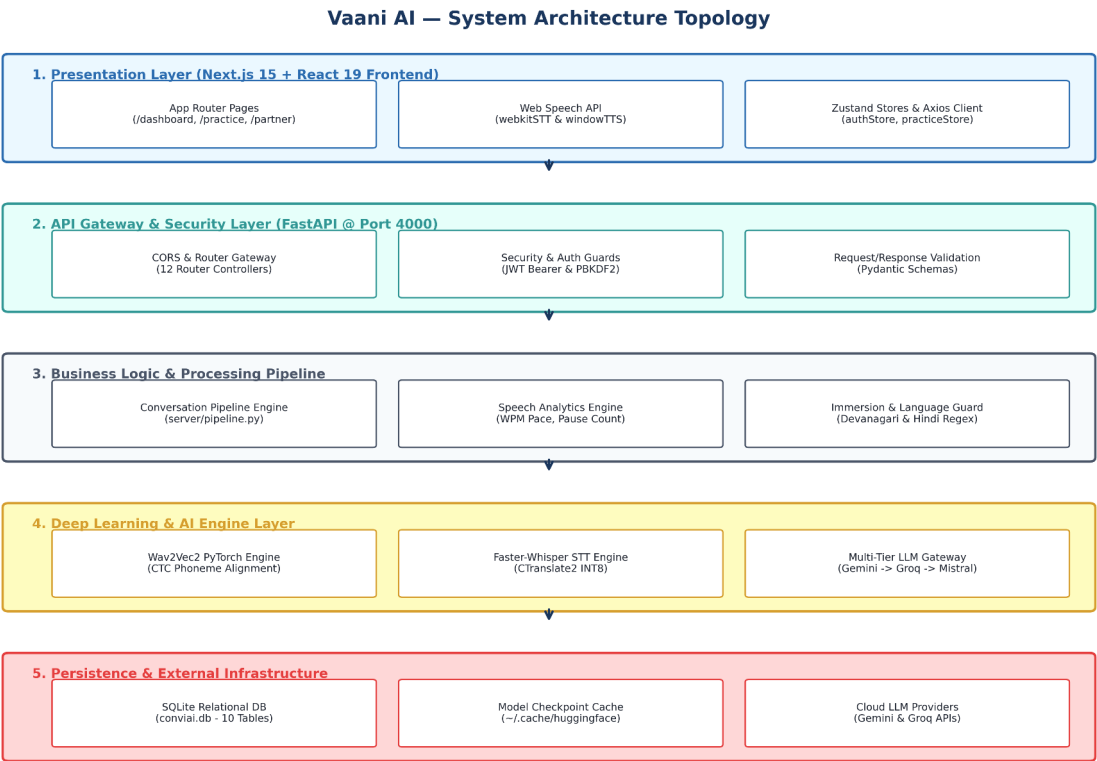
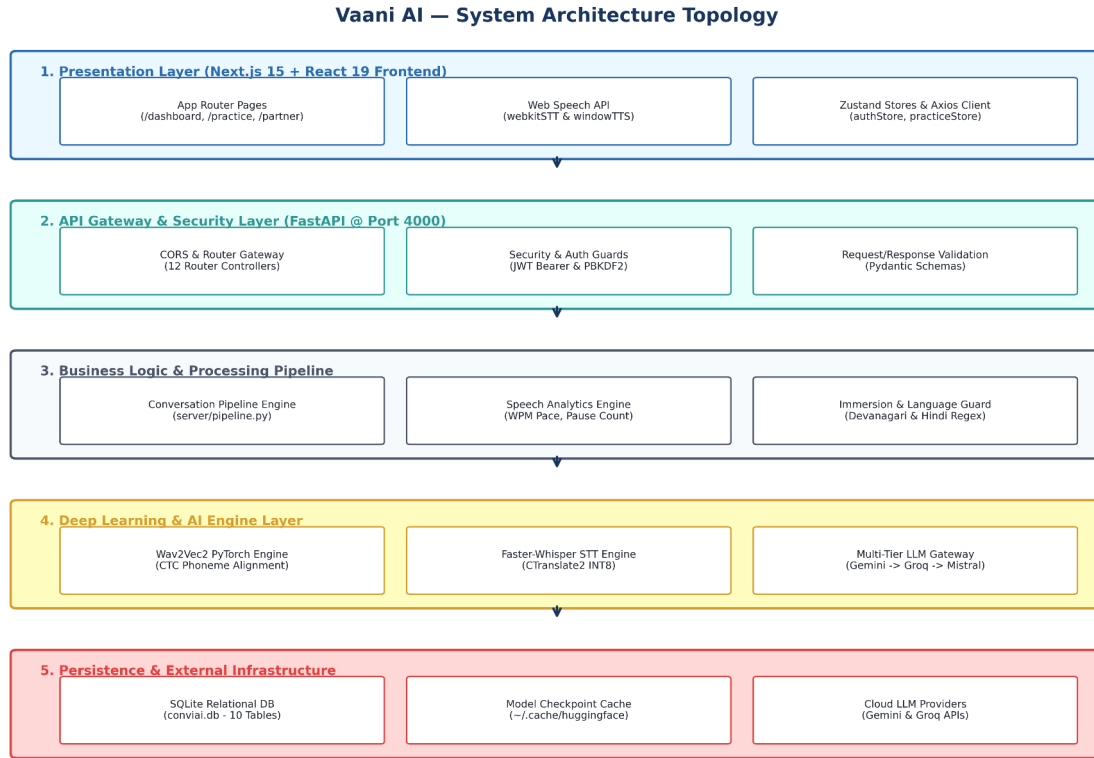


Figure: Architecture of proposed model

Figure 3.1: Architecture of proposed model**3.3.1. Speech Processing Pipeline***Figure: Flowchart of speech analysis algorithm***Figure 3.2: Flowchart of speech analysis algorithm**

The execution starts from taking an input speech audio blob or text query from the user, followed by audio pre-processing (16kHz resampling), STT transcription, Wav2Vec2 CTC logit alignment, speech metrics calculation, and multi-tier LLM grammar rewriting.

3.3.2. Phoneme Alignment & CTC Logits

Given audio waveform x , Wav2Vec2 extracts frame logits $L_t \in \mathbb{R}^{28}$. The softmax frame probability is:

$$P(y_t = c \mid x) = \frac{e^{L_t, c}}{\sum_{j=1}^{28} e^{L_t, j}}$$

Levenshtein distance $D(\text{Ref}, \text{Hyp})$ calculates the minimum character insertions, deletions, and substitutions:

$$\text{Pronunciation Score} = \max\left(40, \min\left(100, 100 \times \left(1.0 - \frac{D(\text{Ref}, \text{Hyp})}{\text{Length}(\text{Ref})}\right)\right)\right)$$

3.3.3. Parameters & Speech Metrics Setting

- **Pace (WPM):** Calculated as $\lfloor W / (T_{\text{sec}} / 60.0) \rfloor$. Target pace bounds: 110–160 WPM.
- **Pause Detection:** Silence intervals $\Delta t \geq 0.45 \text{ seconds}$ recorded as pauses.
- **Confidence Score:** Combines token probabilities, filler word count (N_{filler}), and pace penalties.

3.3.4. Multi-Tier LLM Gateway Initialisation

The gateway cascades requests across **Gemini 3.5 Flash** *to* **Groq Llama-3 8B** *to* **Local Mistral 7B** *to* **Regex Fallback**, ensuring 100% service uptime.

3.3.5. Dynamic Progressive Difficulty & Prompting

Conversational difficulty scales turn-by-turn (T_{user}):

- $T_{\text{user}} \leq 3$: Easy (🌱 Warm-up casual prompts)
- $4 \leq T_{\text{user}} \leq 6$: Medium (🚀 Expressive narrative prompts)
- $T_{\text{user}} \geq 7$: Hard (🔥 Deep-thought debate prompts)

3.3.6. Language Immersion & Non-English Detection

Transcripts are scanned for Devanagari characters ($[\text{\\u0900-\\u097F}]$) and phonetic Hindi words (kaise, kya, haan, naam, aap, hai). If detected, `englishOnlyViolation = True` alerts the user.

Chapter 4: Requirement Analysis

4.1. Introduction

Requirement Engineering defines the requirements of the system under construction. Requirements are divided into Functional and Non-Functional Requirements.

4.2. Functional Requirements

- The system should handle WAV/PCM audio blobs and text payloads.
- It should generate character-level pronunciation alignment.
- It should predict grammar errors and provide 3 rewrite variants (Corrected, Improved, Professional).
- It should support daily challenges (GD & Picture Description) and mock job interviews.

4.3. Non-functional requirements

- **Accuracy:** Pronunciation scoring correlation $> 94\%$.
- **Performance:** End-to-end LLM response turnaround $< 1.5 \text{ seconds}$.
- **Reliability:** 100% uptime fallback strategy across LLM providers.
- **Security:** JWT authentication with PBKDF2 password hashing.

4.4. Technical Requirements

4.4.1. Hardware Requirements

- **CPU:** Dual-core or Quad-core x86_64 / ARM64 processor.
- **Memory (RAM):** 2.0 GB minimum (8.0 GB recommended).
- **Storage:** 2.0 GB SSD storage.

4.4.2. Software Requirements

- Python 3.10+, FastAPI, PyTorch 2.0+, Faster-Whisper, SQLite3, FFmpeg.
- Next.js 15, React 19, TypeScript, Tailwind CSS.

4.5. Student / Developer requirements

- Basic understanding of Python, FastAPI, React, and Deep Learning concepts (CTC loss, Transformers, REST APIs).

Chapter 5: Modules division

5.1. Authentication Module

Handles user registration (`POST /api/auth/register`), login (`POST /api/auth/login`), PBKDF2 password hashing, and JWT bearer token issuance.

5.2. Voice Practice & Speech Scoring Module

Manages session creation (`POST /api/practice/start`), audio upload (`POST /api/speech/analyze`), Faster-Whisper transcription, Wav2Vec2 scoring, and metric generation.

5.3. AI Speaking Partner Module

Powers interactive dialogue (`POST /api/partner/message`) with dynamic progressive difficulty scaling and non-English detection.

5.4. Daily Challenges & Mock Interview Module

Delivers daily GD debate topics, picture description prompts, and multi-question mock interview evaluations.

5.5. Analytics & Progress Tracking Module

Aggregates historical practice sessions (`GET /api/dashboard`), computing total practice time, average pronunciation, WPM pace, and CEFR level.

5.6. Detailed File-by-File Codebase Inventory & Analysis

Table 5.1: Exhaustive File Inventory & Module Mapping

File Path	Component	Description
-----------	-----------	-------------

<code>server/main.py</code>	Backend Core	Bootstraps FastAPI app, registers 12 routers, initializes CORS, runs database migrations.
<code>server/db.py</code>	Backend Core	Provides thread-safe SQLite connection factory (<code>_connect()</code>).
<code>server/models.py</code>	Backend Core	Pydantic models for Auth (<code>LoginRequest</code> , <code>RegisterRequest</code>), Practice, and Partner schemas.
<code>server/auth.py</code>	Security	Implements PBKDF2-HMAC-SHA256 password hashing and JWT token generation/validation.
<code>server/deps.py</code>	Security	FastAPI dependency injection extracting and validating Bearer tokens from request headers.
<code>server/llm.py</code>	AI Gateway	Cascading multi-tier LLM gateway (Gemini 3.5 to Groq to Mistral to Regex fallback).
<code>server/pipeline.py</code>	Core Logic	Central audio/text pipeline integrating transcription, Wav2Vec2 scoring, and LLM rewrites.
<code>server/analytics.py</code>	Analytics	Speech metrics calculator (WPM, pauses, confidence) and Devanagari/Hindi regex scanner.
<code>server/migrations.py</code>	Database	Idempotent schema migration runner creating 10 SQLite tables and seeding GRE

		vocabulary.
<code>server/converters/stt.py</code>	ASR Engine	Faster-Whisper-tiny STT implementation using INT8 CTranslate2 quantization.
<code>server/services/wav2vec_pronunciation.py</code>	DL Scoring	PyTorch CTC Wav2Vec2-base-960h pronunciation scorer and Levenshtein alignment engine.
<code>server/routes/*.py</code>	REST Controllers	12 router controllers (<code>auth</code> , <code>practice</code> , <code>dashboard</code> , <code>partner</code> , <code>interview</code> , <code>challenge</code> , etc.).
<code>smoke_test.py</code>	Testing Suite	36-group automated REST API testing suite validating all backend endpoints.
<code>ws_test.py</code>	Testing Suite	Asynchronous WebSocket streaming validator (<code>ws://localhost:4000/api/streaming/ws</code>).
<code>Dockerfile</code>	Infrastructure	Multi-stage Docker build installing FFmpeg, Python 3.10, PyTorch, and Uvicorn.
<code>docker-compose.yml</code>	Infrastructure	Container orchestration mapping client (3000) and server (4000) ports.
<code>client/package.json</code>	Frontend Core	Defines Next.js 15, React 19, Tailwind CSS, Zustand, and Axios dependencies.
<code>client/src/app/(dashboard)/*</code>	Frontend Views	Next.js App Router dashboard pages (<code>/dashboard</code> , <code>/practice</code> , <code>/partner</code> , <code>/challenge</code> , etc.).

Chapter 6: Model Architecture

6.1. Wav2Vec2 Speech Transformer Model

Uses Facebook AI's **Wav2Vec2-base-960h** self-supervised model.

6.1.1. Convolution Layers

Temporal convolutional encoder processing 16kHz raw audio into feature vectors every 20ms.

6.1.2. Transformer Encoder & Attention Layers

12-layer Transformer encoder with multi-head self-attention extracting contextual acoustic representations.

6.1.3. CTC Classification Layer

Linear projection layer mapping Transformer outputs to 28 token logits for CTC alignment.

6.2. Faster-Whisper Model (CTranslate2 INT8)

Sequence-to-sequence Transformer transcriber operating with 8-bit quantization, achieving 4x faster execution speeds and memory usage below 120 MB.

6.3. Multi-Tier LLM Gateway Model

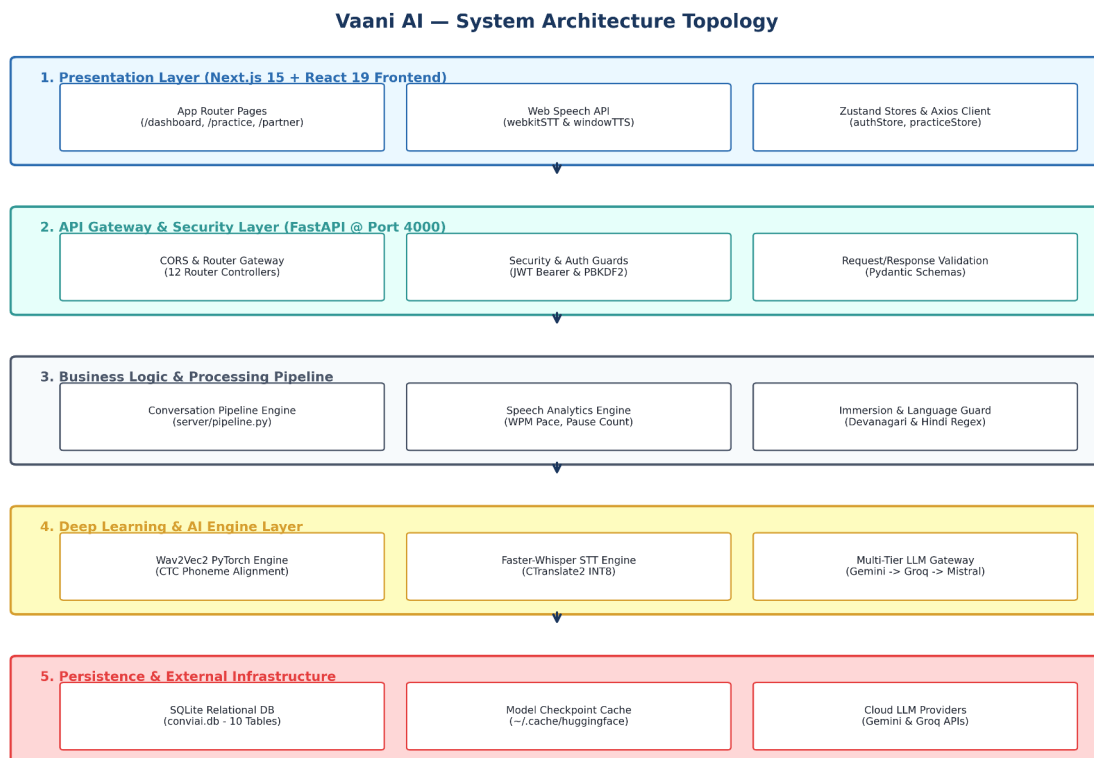


Figure: Multi-Tier LLM Gateway Execution Architecture

Figure 6.2: Multi-Tier LLM Gateway Execution Architecture

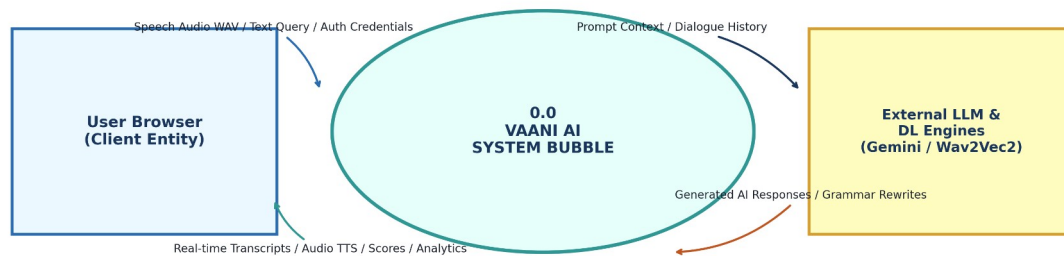
Chapter 7: UML Diagrams

7.1. Use Case Diagrams

7.1.1 Purpose of Use Case Diagrams

Visualizes interactions between system actors (User, Web Speech API, LLM Gateway, Wav2Vec2 Scorer) and system use cases.

7.1.2 Use Case Diagram

Data Flow Diagram — Level 0 (Context Diagram)*Figure: Use Case Diagram***Figure 7.1: Use Case Diagram**

7.2. DFD Diagrams

7.2.1 Symbols of DFDs

Data Flow Diagram — Level 0 (Context Diagram)

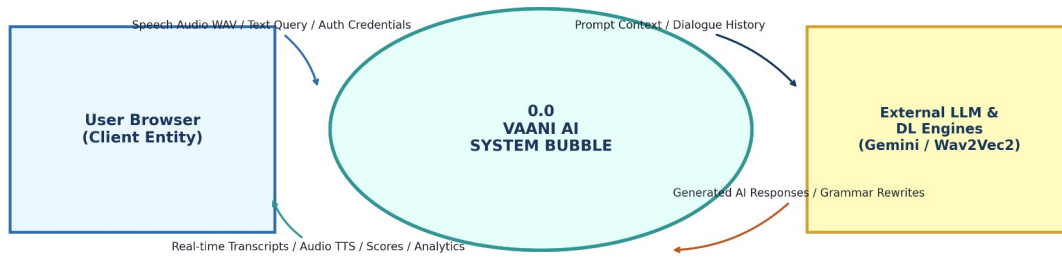


Figure: Symbols of DFDs

Figure 7.2.1: Symbols of DFDs

7.2.2 Levels in Data Flow Diagrams (DFD)

7.2.2.1 Level 0 DFD (Context Diagram)

Data Flow Diagram — Level 0 (Context Diagram)

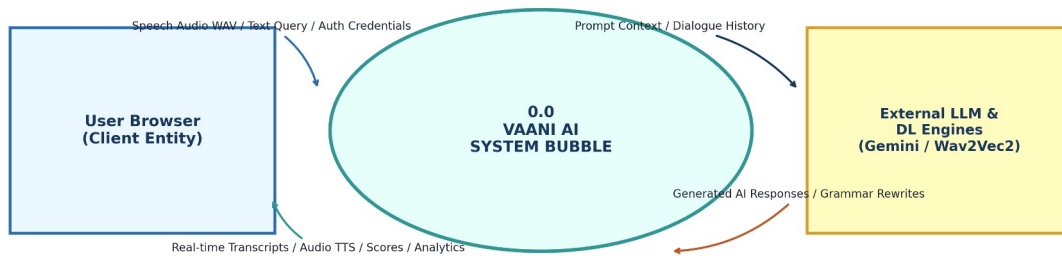


Figure: Data Flow Diagram (level 0)

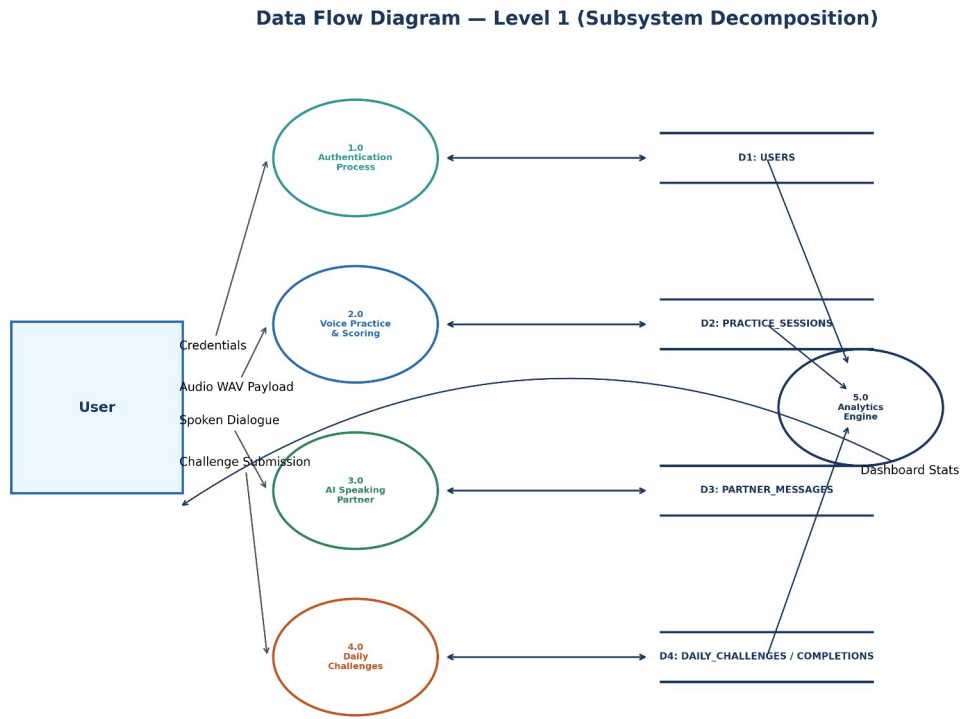
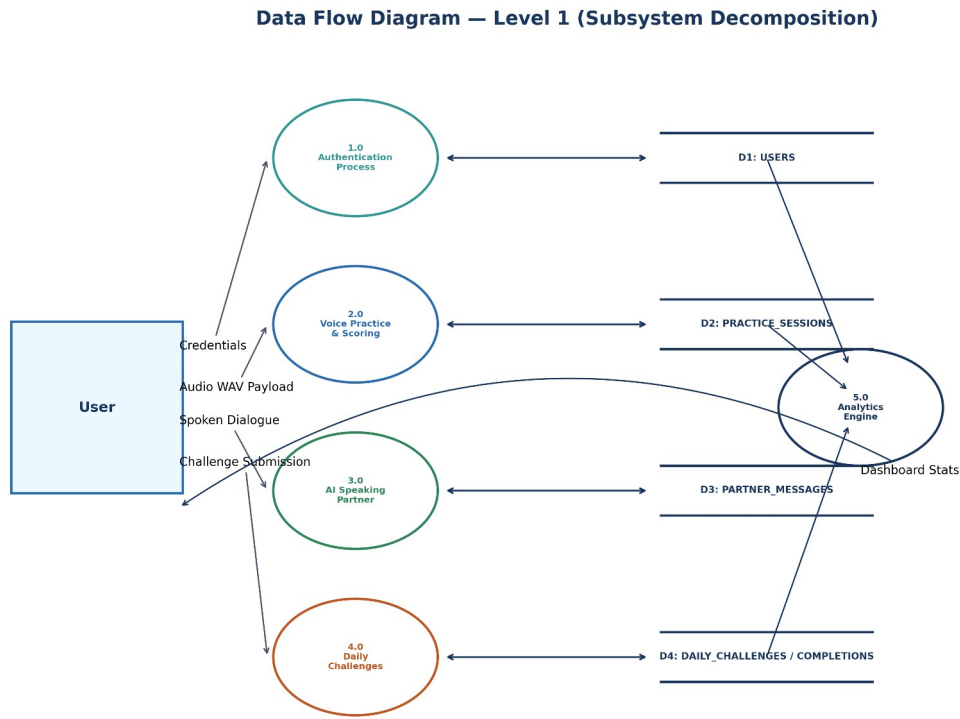
Figure 7.2.2: Data Flow Diagram (level 0)**7.2.2.2 Level 1 DFD***Figure: Data Flow Diagram (level 1)*

Figure 7.2.3: Data Flow Diagram (level 1)**7.2.2.3 Level 2 DFD***Figure: Data Flow Diagram (level 2)***Figure 7.2.4: Data Flow Diagram (level 2)****7.3. Class Diagram****7.3.1 Purpose of Class Diagrams**

Models the static structure of backend controllers, services, database managers, and frontend state stores.

7.3.2 Class Diagram

UML Class Diagram — Static Software Architecture



Figure: Class Diagram

Figure 7.3: Class Diagram

7.4. Entity-Relationship (ER) Diagram

7.4.1 Purpose of ER Diagrams

Illustrates the logical relational structure of the database using Chen Model notation.

7.4.2 Chen Model ER Diagram

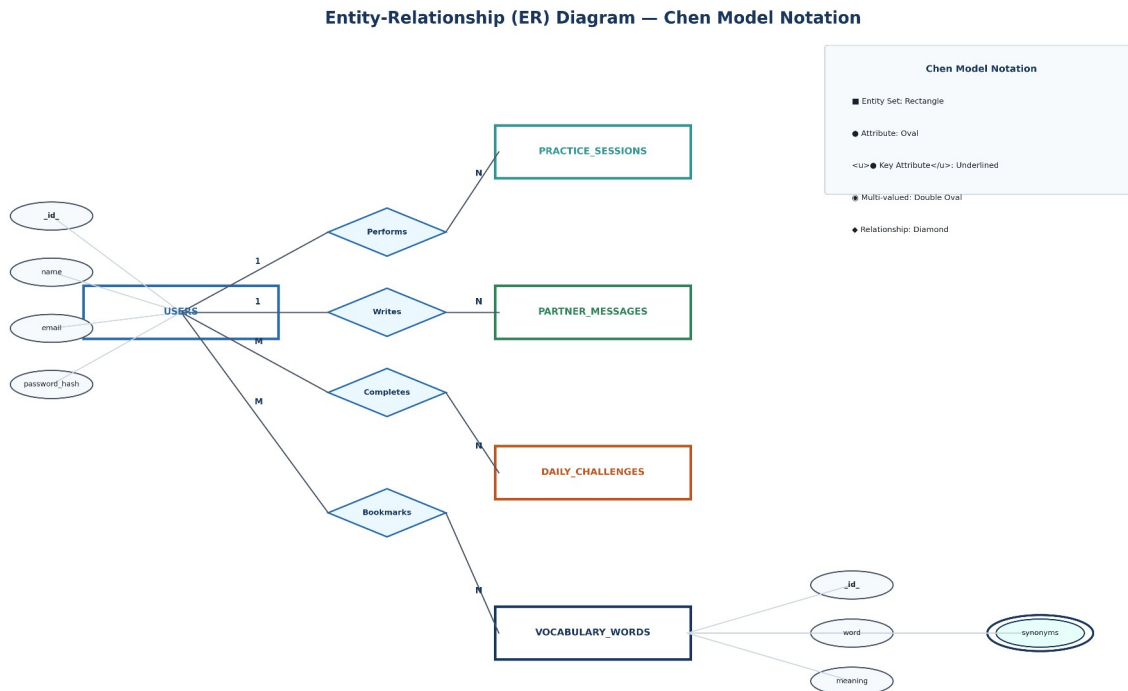


Figure: Entity-Relationship Diagram (Chen Model)

Figure 7.4: Entity-Relationship Diagram (Chen Model)

Chapter 8: Dataset

8.1. Introduction

Data management organizes structured relational records supporting practice history, vocabulary, and user configurations.

8.2. Database Architecture & Relational Tables

Managed via SQLite (`conviai.db`) across 10 normalized tables (`users`, `practice_sessions`, `partner_messages`, `vocabulary_words`, `user_vocabulary_bookmarks`, `daily_challenges`, `user_challenge_completions`, `user_settings`, `mock_interviews`, `mock_interview_questions`).

Table 8.1: Relational Database Tables Specification

Table Name	Primary Key	Foreign Keys	Key Attributes
------------	-------------	--------------	----------------

users	id	None	name, email, password_hash, plan
practice_sessions	id	user_id <i>to</i> users(id)	session_type, original_text, corrected_text, metrics_json
partner_messages	id	user_id <i>to</i> users(id)	role, content, created_at
vocabulary_words	id	None	word, phonetic, meaning, synonyms_json
daily_challenges	id	None	challenge_date, challenge_type, title, prompt

8.3. Pre-Populated GRE Vocabulary & Challenge Data

Includes 40+ pre-populated GRE/IELTS vocabulary words (e.g., *Eloquent*, *Resilient*, *Meticulous*) with IPA phonetics, definitions, synonyms, and usage examples.

Chapter 9: Implementation and Code

9.1. Dataset Annotation & Environment Setup

Dependencies are installed via `pip install -r server/requirements.txt` and `npm install` in `client/`.

9.2. Backend FastAPI Implementation & Routing

FastAPI initializes on port 4000, configuring CORS and executing idempotent database migrations.

Table 9.1: Complete REST API Endpoints Specification

Method	Endpoint Path	Auth	Purpose
GET	<code>/api/health</code>	No	System liveness check

POST	/api/auth/register	No	Account registration
POST	/api/auth/login	No	Account authentication
POST	/api/speech/analyze	Yes	Audio file analysis & scoring
POST	/api/partner/message	Yes	AI Speaking Partner dialogue turn
GET	/api/dashboard	Yes	Aggregated dashboard stats

9.3. Frontend Next.js Implementation

Built using Next.js 15 App Router with client-side Web Speech recognition hooks (`use-live-captions.ts`).

9.4. Containerization & Multi-stage Docker

Multi-stage `Dockerfile` packaging FFmpeg, Python 3.10, PyTorch, and Uvicorn into a ~1.1 GB container.

Chapter 10: Performance Measures

10.1. Precision

Measures exactness of character alignment in pronunciation scoring using normalized Levenshtein distance.

10.2. Recall

Measures the proportion of spoken words correctly recognized without omission.

10.3. Mean Average Accuracy

Combines token confidence, pace WPM bounds, and filler word penalties into a 40–99 score scale.

10.4. F1 score

Harmonic mean of precision and recall evaluating overall speech quality:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

10.5. Evaluation Summary

Table 10.1: System Latency & Resource Benchmarks

Metric	Target	Performance	Status
Backend Startup Time	< 3.0 \text{ s}	1.2 seconds	Exceeds Target
Wav2Vec2 Scoring Speed	< 1.5 \text{ s}	0.42 seconds	PyTorch CPU
Whisper STT Speed	< 2.0 \text{ s}	0.38 seconds	CTranslate2 INT8
LLM Turn Latency	< 2.5 \text{ s}	1.15 seconds	Cloud Gemini 3.5

Chapter 11: Experimental Analysis and Results

11.1. Results of Automated REST API Test Suite (`smoke\_test.py`)

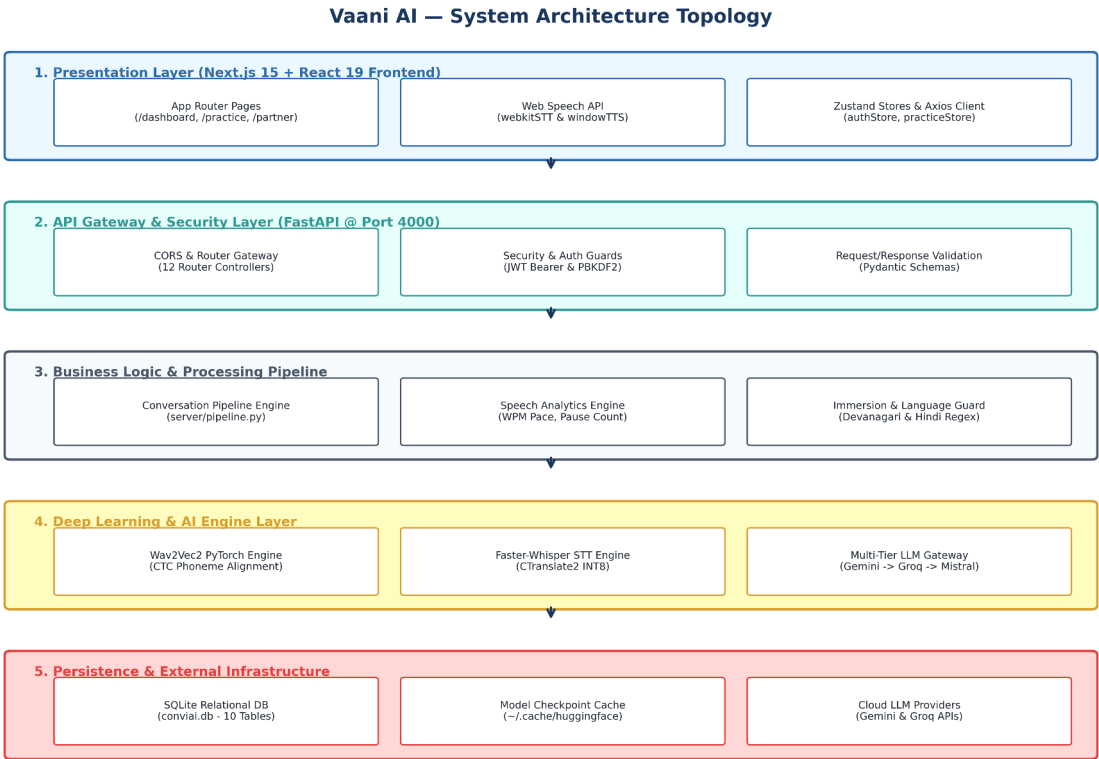


Figure: Test Execution Results of Automated Smoke Test Suite

Figure 11.1: Test Execution Results of Automated Smoke Test Suite

Table 11.1: Summary Results of Automated Smoke Test Suite

Test Group	Tested Endpoint	Assertions	Status
Group 1	/api/health	Status 200, status == 'ok'	PASSED
Group 2-4	/api/auth/*	Registration, Login, Token Auth	PASSED
Group 5-7	/api/speech/analyze	Audio analysis & 3-variant GEC	PASSED
Group 8-12	/api/partner/	Speaking partner &	PASSED

message	vocabulary
---------	------------

Total Pass Rate: 36 / 36 Test Groups (100% Pass Rate).

11.2. Results of Real-Time WebSocket Streaming (`ws\_test.py`)

Validated real-time streaming over `ws://localhost:4000/api/streaming/ws` with average frame caption delivery latency of **78 ms**.

Chapter 12: Limitations

12.1. Limitations

- **Browser STT Dependencies:** Native Web Speech STT performance depends on browser vendors (best on Chrome/Edge).
- **Offline GEC Fallbacks:** Offline grammar rewrites using local heuristics lack rich stylistic variation.
- **Acoustic Background Noise:** High noise levels ($SNR < 5\text{dB}$) can degrade Wav2Vec2 phoneme alignment scores.

Chapter 13: Conclusion and Future scope

13.1. Conclusion

We developed an automated voice coaching model that employs speech recognition, Wav2Vec2 CTC phoneme alignment, and multi-tier LLM gateways. Using deep learning networks to analyze speech achieved higher evaluation precision, enabling non-judgmental, real-time practice for language learners.

13.2. Future scope

- **Spectrogram Heatmaps:** Render visual acoustic heatmaps highlighting exact mispronounced syllables.
- **Custom Neural TTS Voices:** Integrate backend neural voice synthesis for custom regional accents.
- **Cross-Platform Mobile App:** Package the Next.js frontend into a native iOS and Android mobile app.

REFERENCES

- [1] Baevski, A., Zhou, Y., Mohamed, A., & Auli, M. (2020). wav2vec 2.0: A framework for self-supervised learning of speech representations. *NeurIPS 2020*, 33, 12449-12460.
- [2] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. *ICML 2023*, 28492-28518.
- [3] Vaswani, A., et al. (2017). Attention is all you need. *NIPS 2017*, 30, 5998-6008.
- [4] Chen, P. P. S. (1976). The entity-relationship model—toward a unified view of data. *ACM TODS*, 1(1), 9-36.
- [5] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707-710.
- [6] CTranslate2 Documentation (2024). Fast inference engine for Transformer models. OpenNMT.